Below is a **real-world, day-to-day used master list of Spring Framework classes + annotations + methods** that you will commonly use in **enterprise applications** across **Spring Framework (3 → 6+) + Spring Boot ecosystem**, with **real-time use cases + examples**.

✅I'm focusing on **MOST USED in practical development** (not rare/internal classes).

---

1) Core Spring Container (IoC / DI) – Common Classes & Methods

Category	Class / Interface	Common Methods	Real-time Use Case	Example (real)
IoC Container	`ApplicationContext`	`getBean()`, `containsBean()`, `getEnvironment()`	Access Spring container, fetch beans	Load service bean in integration test
Bean Factory	`BeanFactory`	`getBean()`, `isSingleton()`	Low-level DI container	Used in frameworks or libraries
Context bootstrap	`AnnotationConfigApplicationContext`	`register()`, `refresh()`, `close()`	Run Spring without Spring Boot	Desktop app / CLI app
Bean scope	`ConfigurableBeanFactory`	`SCOPE_SINGLETON`, `SCOPE_PROTOTYPE`	Configure bean scopes	Prototype objects per request
Bean registration	`BeanDefinitionRegistry`	`registerBeanDefinition()`	Dynamic bean registration	Multi-tenant dynamic modules
Aware interfaces	`ApplicationContextAware`	`setApplicationContext()`	Get context injected by Spring	Utility beans needing context
Environment	`Environment`	`getProperty()`, `acceptsProfiles()`	Read config values and profiles	`dev`, `prod` switching
Profiles	`Profiles`	`of()`	Conditiona	Enable

Category	Class / Interface	Common Methods	Real-time Use Case	Example (real)
			l beans	bean for prod only
Property source	`PropertySource`	`getProperty()`	Custom config sources	Read config from DB
Lifecycle	`SmartLifecycle`	`start()`, `stop()`, `isRunning()`	Start/stop services automatically	Kafka listener startup

---

2) Most Used Core Spring Annotations (DI / Configuration)

Annotation	Used On	Real Use Case	Real Example
`@Component`	Class	Generic bean	Helper / util service
`@Service`	Class	Service layer bean	`UserService`
`@Repository`	Class	DAO/repository bean	DB operations + exception translation
`@Configuration`	Class	Config class	`@Bean` definitions
`@Bean`	Method	Create beans manually	`RestTemplate`, `ObjectMapper`
`@Autowired`	Field/Constructor	Inject dependencies	Inject repository into service
`@Qualifier`	Parameter	Choose correct bean	2 `DataSource` beans
`@Primary`	Bean	Default bean selection	default `ObjectMapper`
`@Value`	Field/Parameter	Inject property value	`${jwt.secret}`
`@PropertySource`	Config class	load `.properties` file	custom property files
`@Lazy`	bean injection	delay bean init	heavy init beans
`@Scope`	Class / bean	prototype/request scope	request scoped bean

---

3) Spring Bean Validation (Hibernate Validator Integration)

Class / Annotation	Common Methods	Use Case	Example
`@Valid`	—	Validate DTO in REST	Validate signup request
`@Validated`	—	Validation in service layer	validate method params

Class / Annotation	Common Methods	Use Case	Example
`BindingResult`	`hasErrors()`, `getFieldErrors()`	read validation errors	show messages
`ConstraintViolationException`	`getConstraintViolations()`	service validation errors	fail fast

Example:

```
@PostMapping("/users")
public ResponseEntity<?> create(@Valid @RequestBody UserDto dto) {
    return ResponseEntity.ok(service.create(dto));
}
```

---

4) Spring AOP (Cross-cutting concerns)

Class / Annotation	Common Methods	Real Use Case	Example
`@Aspect`	—	define aspect	logging/tracing
`@Around`	join point advice	execution time logging	measure API response time
`ProceedingJoinPoint`	`proceed()`, `getArgs()`	execute method in around advice	custom audit

Example:

```
@Around("execution(* com.app.service..*(..))")
public Object log(ProceedingJoinPoint pjp) throws Throwable {
  long start = System.currentTimeMillis();
  Object result = pjp.proceed();
  System.out.println("Time=" + (System.currentTimeMillis() - start));
  return result;
}
```

---

5) Spring Transaction Management (Highly used)

Class / Annotation	Common Methods	Use Case	Example
`@Transactional`	—	DB transaction	money transfer
`TransactionTemplate`	`execute()`	programmatic transaction	partial rollback logic
`PlatformTransactionManager`	`getTransaction()`, `commit()`, `rollback()`	low-level tx mgmt	custom transactions

Class / Annotation	Common Methods	Use Case	Example
`DataAccessException`	—	runtime db exceptions	avoid boilerplate

Example:

```
@Transactional
public void transfer(Long from, Long to, BigDecimal amount) {
    debit(from, amount);
    credit(to, amount);
}
```

---

6) Spring JDBC Template (Still widely used)

Class	Common Methods	Use Case	Example
`JdbcTemplate`	`query()`, `queryForObject()`, `update()`	direct SQL CRUD	Reports, legacy DB
`NamedParameterJdbcTemplate`	`query()`, `update()`	SQL with named params	large queries
`RowMapper<T>`	`mapRow()`	map resultset	DB → object

---

7) Spring Web MVC (Controllers / REST APIs)

Category	Class / Annotation	Common Methods	Real Use Case	Example
REST Controller	`@RestController`	—	REST API	`/api/users`
Routing	`@RequestMapping`	—	base route mapping	`/api/v1`
HTTP mapping	`@GetMapping`, `@PostMapping`, etc.	—	endpoint definitions	user CRUD
Request parsing	`@RequestBody`	—	JSON input	signup payload
Params	`@RequestParam`	—	query params	`/search?q=abc`
Path vars	`@PathVariable`	—	URL parameters	`/users/{id}`
Headers	`@RequestHeader`	—	read token header	`Authorization`
Response	`ResponseEntity`	`ok()`, `status()`	proper REST	201/400/404

Category	Class / Annotation	Common Methods	Real Use Case	Example
			responses	
Exception handling	`@ControllerAdvice`	—	global error handler	custom error JSON
Error mapping	`@ExceptionHandler`	—	map exceptions	`UserNotFoundException`
Model/View	`Model`	`addAttribute()`	SSR apps	Thymeleaf UI

Example:

```
@GetMapping("/users/{id}")
public ResponseEntity<UserDto> get(@PathVariable Long id) {
    return ResponseEntity.ok(service.get(id));
}
```

---

8) Spring Filters & Interceptors (Very practical)

Component	Class	Key Methods	Use Case
Servlet Filter	`OncePerRequestFilter`	`doFilterInternal()`	JWT authentication
Interceptor	`HandlerInterceptor`	`preHandle()`, `postHandle()`	logging, rate limit
Config	`WebMvcConfigurer`	`addInterceptors()`, `addCorsMappings()`	CORS + interceptors

---

9) Spring REST Client (Outgoing calls)

Class	Methods	Use Case	Notes
`RestTemplate`	`getForObject()`, `postForEntity()`, `exchange()`	call external services	legacy but used
`WebClient` *(Spring 5+)*	`get()`, `post()`, `retrieve()`, `bodyToMono()`	reactive rest calls	recommended now

Example (WebClient):

```
webClient.get()
  .uri("/users/{id}", id)
  .retrieve()
  .bodyToMono(UserDto.class);
```

---

10) Spring Async & Scheduling

Annotation/Class	Common Methods	Use Case	Example
`@Async`	—	run async methods	send email async
`@EnableAsync`	—	enable async	config
`@Scheduled`	—	cron-based tasks	daily report
`ThreadPoolTaskExecutor`	`setCorePoolSize()`	executor config	tune performance

---

11) Spring Events (Decoupled communication)

Class/Annotation	Methods	Use Case
`ApplicationEventPublisher`	`publishEvent()`	publish events
`@EventListener`	—	listen to events
`@TransactionalEventListener`	—	event after commit

Example:

```
publisher.publishEvent(new UserCreatedEvent(userId));
```

---

12) Spring Security (Most used daily)

Component	Class / Annotation	Common Methods	Use Case
Config	`SecurityFilterChain`	`http.authorizeHttpRequests()`	secure APIs
Password	`PasswordEncoder`	`encode()`, `matches()`	store passwords
Authentication	`AuthenticationManager`	`authenticate()`	login
Context	`SecurityContextHolder`	`getContext()`	current user info
Role-based	`@PreAuthorize`	—	RBAC
User details	`UserDetailsService`	`loadUserByUsername()`	load user from DB
JWT filter	`OncePerRequestFilter`	`doFilterInternal()`	JWT validation

Example:

```
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/users/{id}")
public void delete(@PathVariable Long id){ service.delete(id); }
```

---

13) Spring Data JPA (Most common database layer)

Interface/Class	Common Methods	Real Use Case
`JpaRepository<T,ID>`	`save()`, `findById()`, `findAll()`, `deleteById()`	CRUD
`CrudRepository`	basic CRUD	minimal apps
`PagingAndSortingRepository`	`findAll(Pageable)`	pagination
`Pageable`, `PageRequest`	`of()`	paging
`Page<T>`	`getContent()`, `getTotalElements()`	API pagination
`@Entity`	—	DB entity
`@Id`, `@GeneratedValue`	—	primary key
`@OneToMany`, `@ManyToOne`	—	relationships
`@Query`	—	custom query
`Specification<T>`	`toPredicate()`	dynamic filtering

Example:

```
public interface UserRepo extends JpaRepository<User, Long> {
   Optional<User> findByEmail(String email);
}
```

---

14) Testing (Spring Test, JUnit)

Annotation/Class	Key Use	Real Example
`@SpringBootTest`	full context test	integration tests
`@WebMvcTest`	controller-only test	REST controller test
`MockMvc`	simulate HTTP calls	`perform(get("/users"))`
`@MockBean`	mock spring bean	mock service layer

✅Conclusion: What you use MOST in daily Spring projects

Most used in every project:

- `@RestController`, `@GetMapping`, `@PostMapping`
- `@Service`, `@Repository`, `@Autowired`
- `@Transactional`
- `ResponseEntity`
- `JpaRepository` (`save`, `findById`, `findAll`, `deleteById`)
- `@Valid`

- Spring Security (`SecurityFilterChain`, `PasswordEncoder`, `@PreAuthorize`)
- `@ControllerAdvice` + `@ExceptionHandler`

If you want, I can provide the **same list in an even bigger master table (200+ classes & annotations)** including:
✅Spring Cloud (Config, Gateway, Eureka)
✅Kafka / RabbitMQ integrations
✅Spring Batch
✅Spring WebFlux (Reactive)
✅Actuator & Observability (Micrometer)